Raft and Fault-Tolerant Key/Value Storage

Everton Romanzini Colombo
e257234@dac.unicamp.br

Student 2
student2@unicamp.br

I. Introduction

Replicating a service across several machines is the standard way to make systems resilient to machine crashes, but copying alone is not enough: the replicas must agree on a single order of operations, and they must keep agreeing while links drop messages, nodes leave and reenter the grid, leaders disappear, among other failures. Many consensus algorithms that provide a solution to this problem exist today, and maintaining a log of agreed-upon state-machine operations is a common implementation approach. The Raft algorithm [1] is a widely used consensus protocol that is designed to be understandable and easy to teach, and it is the basis of many production systems, including etcd, Consul, and CockroachDB **[Everton:** cite]. The practical question, however, is not only whether a protocol can be implemented correctly, but what that correctness costs in throughput, tail latency, and recovery time when faults actually happen.

With this in mind, this project had two main elements: 1. Implementing Raft, as well as a simple key-value store system on top of it, with the goal of understanding in detail how this consensus algorithm works; and 2. Building an experimental harness on top of these implementations to measure the cost of fault tolerance in a realistic environment. By implementing the resulting system, key distributed systems concepts were explored, such as leader election, consensus, replication and consistency, fault detection and tolerance, peer-to-peer communication, among others.

Basis and contributions

The starting point for this project was the framework supplied by MIT's 6.824 (2020) course labs 2 and 3 [2]. It provided stub APIs, an in-memory simulated network (`labrpc`), a gob (Go's serialization library) wrapper (`labgob`), the Porcupine linearizability checker [3], and, crucially, a comprehensive and thorough set of tests. This skeleton was kept intact and was used to guide and verify our implementation. On the other hand, the protocol and service implementations, the runtime communication setup, persistence, container packaging, client interface, and the whole experimental apparatus were written by us. Concretely, this project's contributions are:

- **In-House Raft Implementation:** following MIT 6.824's labs 2A–2C + 3B, our custom raft implementation closely follows Raft's extended paper guidelines, including leader election with randomized timeouts, the log up-to-date voting rule, log replication with fast backup, safety rules for commitment across terms, persistent state restored after a crash, and log compaction with snapshots.
- **Fault-tolerant Key/Value Service Implementation:** following MIT 6.824's labs 3A–3B, a replicated key-value store service was built on top of the Raft implementation built before. It includes client-side leader discovery with retry, response caching, and state snapshotting.
- **Deployment Setup**: the systems' transport layer, ports, and persistence behavior were made configurable via environment variables, so that the same code can run both in the labs' test suites (over the simulated `labrpc` network) and in a containerized deployment (over real `net/rpc`/TCP). A Dockerfile for container image configuration, and a `compose.yaml` file for setting up multiple nodes were provided.
- **Interface and Experimental Harness**: an HTTP/JSON API was exposed so that clients can interface with the KV application. A `/status` endpoint was also created, exposing term, leader, commit index, last log index, applied index and /snapshot sizes, plus a benchmark client and a scenario-driven experiment driver (normal operation, network delay/loss, leader crash, partition, cluster-size sweep) that record raw CSVs and check every history for linearizability with Porcupine.

[TODO: Check that this contribution list matches the emphasis of the abstract and of the five-minute video, and trim it if the six-page limit becomes tight.]

The remainder of this report is organized as follows. Section III states the system model and the failure assumptions. Section IV describes the architecture and the container deployment. Section V details the two algorithms and lists the reused code and our modifications. Section VI argues the correctness properties and describes the automated tests. Section VII defines the experimental methodology and its reproducibility, Section VIII presents the results, Section IX discusses limitations and threats to validity, and Section X concludes.

II. Background: Raft

A common way to build a fault-tolerant service is the *replicated state machine* [4]: every server keeps its own copy of the service state (in our case, a key/value map) and a log of commands. If all servers apply the same commands in the same order, all copies go through the same states and end up equal, so any of them can answer for the service (Figure 1). The hard part is getting the servers to agree on the contents of the log while messages get lost and servers crash, which is a consensus problem. Raft [1] solves it for a fixed group of N servers, and keeps working as long as a majority of them (more

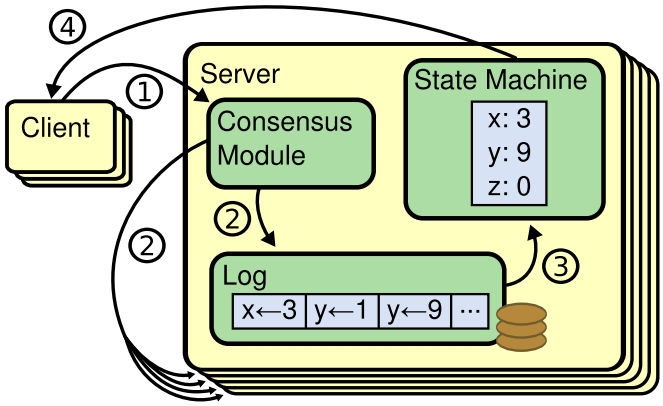


Figure 1. Replicated state machine architecture: the consensus module keeps a replicated log, and each state machine applies the same commands in the same order. Adapted from Figure 1 of [1].

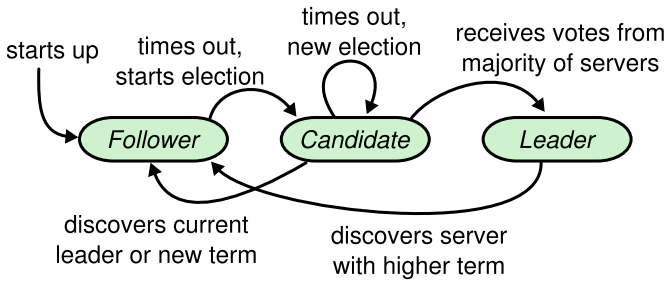


Figure 2. Raft server states and transitions. Adapted from Figure 4 of [1].

than $N/2$) is up and able to talk to each other. A cluster of three servers survives one failure, and a cluster of five survives two.

a) Roles and terms: At any moment, each server is a *follower*, a *candidate* or the *leader* (Figure 2). In normal operation there is exactly one leader, and the other servers are followers that only answer its requests. Time is divided into *terms*, numbered with consecutive integers; each term starts with an election and has at most one leader. Every message carries the sender's term, a server that sees a term higher than its own adopts it and becomes a follower, and messages from older terms are rejected. This is how a leader that was cut off finds out that it has been replaced. The basic protocol needs only two RPCs, `RequestVote` and `AppendEntries`. Optionally, a `InstallSnapshot` RPC is defined for log compaction.

b) Leader election: The leader sends periodic heartbeats (`AppendEntries` calls, possibly empty) to every follower. A follower that hears nothing for an *election timeout* assumes the leader has failed: it increments its term, becomes a candidate, votes for itself and asks the other servers for their votes. Each server gives at most one vote per term, to the first candidate that asks, and only if that candidate's log is at least as up to date as its own. A candidate that gets votes from a majority becomes the leader and starts sending heartbeats. If the votes are split and nobody wins, the candidates time out and try again in a new term. Election timeouts are chosen at random, so this shouldn't happen many times in a row.

c) Log replication: Clients send their commands to the leader, which appends each one to its log as a new entry tagged with the current term, and sends it to the followers with `AppendEntries`. Each call also carries the index and term of the entry just before the new ones, and a follower only accepts it if its own log has a matching entry at that position. If not, the follower refuses, the leader retries from an earlier position, and once the two logs match, the follower's conflicting entries are replaced by the leader's. When an entry is stored on a majority of servers, the leader considers it *committed*, applies it to its state machine and answers the client service. Followers learn the new commit index from later `AppendEntries` calls and apply the entry too.

d) Safety: Two extra rules keep committed entries from being lost when the leader changes. The voting rule above means that only a candidate that holds every committed entry can win an election. And a leader only declares an entry committed by counting replicas if the entry is from its own term; entries from older terms are committed indirectly, together with a newer one. Figure 8 of the Raft paper shows how a committed entry could be overwritten without this second rule. Section VI-A explains why these rules are enough.

e) Persistence and snapshots: Each server saves its current term, its vote and its log to stable storage before answering any RPC, so that it can crash, restart and rejoin without breaking promises it made before the crash. Since the log would otherwise grow forever, the service periodically saves a *snapshot* of its state, and Raft throws away the log entries that the snapshot covers. A follower that is so far behind that the leader no longer has the entries it needs receives the leader's snapshot through `InstallSnapshot`.

III. System Model

We consider a fixed set of N servers that talk to each other, plus any number of clients. These server nodes are both KV-store servers and Raft peers. Clients come and go and only send requests. We make no assumptions about how fast nodes run or about shared clocks. The assumptions listed here are the ones we rely on when arguing correctness and progress in Section VI.

A. Timing model

Raft is partially synchronous. It does not know in advance any hard limit on message delay or on how long a node takes to execute a step, but timeouts and heartbeats managed by local timers are used to detect failures and guarantee progress.

Safety does not depend on timing at all. The rules for electing a leader, matching logs and committing entries use only the contents of messages and the fact that majorities intersect, so no timeout can make two replicas disagree. Progress does depend on timeout constants, namely: heartbeats every 150 ms, election timeouts of 500–700 ms, peer RPCs bounded at 800 ms and client requests at 2 s.

B. Channels

Each pair of nodes has its own connection, and messages are addressed to one specific node and there is no broadcast medium. Raft communication happens via remote procedure calls, and client-server interaction with the KV-store service layer is done with HTTP requests. We assume that a channel can lose, delay, duplicate and reorder messages, and that a message may take arbitrarily long or never arrive. Retransmission and retries are our job, not the channel's. In the deployed system the connection is TCP carrying `net/rpc` with gob encoding, and we inject loss and delay at the application level; in the test suite the same protocol runs over `labrpc`, which already simulates loss, reordering and delay. Since requests and replies can be duplicated, every operation is idempotent: clients tag each request with a `(ClientId, Seq)` pair and servers answer repeated requests from a cache instead of applying them again.

C. Failures

It is assumed that a node can crash and later restart, losing its volatile state but keeping what it wrote to disk. The network can also split into partitions, for a while or forever. We do not consider Byzantine faults; messages are not forged or corrupted, apart from what the serialization library's decoding can detect. Raft guarantees correctness in (and thus limits progress to) situations in which, with N replicas, no more than $\lfloor (N-1)/2 \rfloor$ nodes fail concurrently.

D. Clocks, persistence and restart

Raft does not assume synchronized clocks, and no algorithm step reads wall-clock time; a local monotonic clock is used only to measure timeouts. Before a node replies to a request or acts on one, it writes to disk what it needs after a crash: the current term, the vote, the log and the last included snapshot index. Each write goes to a temporary file that is then renamed, so a crash in the middle of a write cannot leave truncated state. On restart the node reloads those files and resets only its volatile state (role, leader hint and timers), coming back as a follower.

E. Correctness properties

The system is correct when two kinds of properties hold:

- *Safety:* there is at most one leader per term; two logs that share an entry at the same index also share the whole prefix before it (Log Matching); an entry committed at one replica eventually appears, with the same index and command, at every replica; and the KV service is linearizable, meaning that every read and write looks atomic, taking effect at some instant between its call and its reply.
- *Liveness:* once the network is stable and a majority of nodes is up, a leader is elected after finitely many elections and every command submitted to it is eventually committed and applied.

Section VI argues safety and describes the tests that exercise it.

IV. Architecture

Every node runs the same Go binary (`cmd/kvnode`), with one container per node, orchestrated by Docker Compose (Figure 3). Inside each container a single process hosts both the `KVServer` and its `Raft` peer. The two share memory and exchange messages through the `applyCh` channel and `Raft.Start`, while different nodes talk to each other only over the network. Each node therefore opens two listeners: a TCP port for the internal Raft RPCs and an HTTP port for the client-facing key/value API.

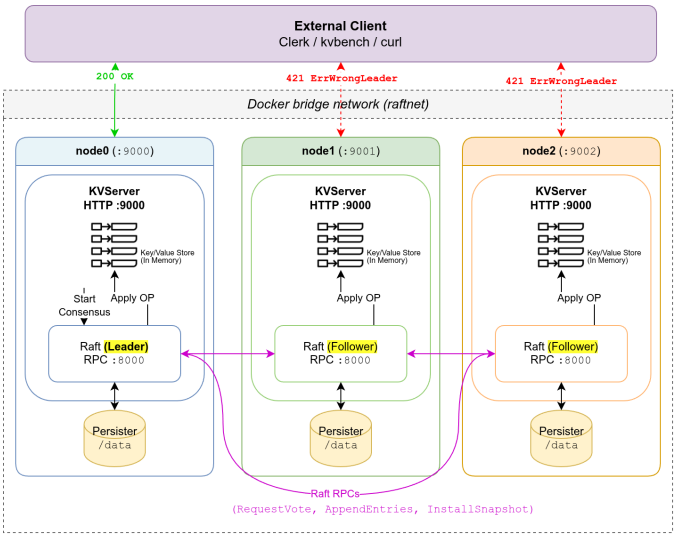


Figure 3. Containerized cluster architecture diagram.

The nodes are configured through environment variables. `NODE_ID` selects the node's own index into `PEERS`, which lists the Raft addresses of every peer in index order; `RAFT_ADDR` and `HTTP_ADDR` set the two listeners; `DATA_DIR` points at the mounted volume; and `MAX_RAFT_STATE` sets the snapshot threshold. See the `compose.yaml` file on this project's repository for an example of how to set these variables for a cluster of three nodes.

This configurability also allows for the same code to run unchanged between the test harness and the container setup: in the tests the nodes are wired over the simulated `labrpc` network, while in the deployment they are wired over real TCP, and the difference is entirely in how the peer endpoints are created. In the code, this is achieved by having Raft and the KV service depend only on a one-method interface for communication, `RPCClient`, which both `labrpc.ClientEnd` (the in-memory simulated network) and `labrpc.TCPEnd` (a `net/rpc` client over TCP with a dial and a call deadline) implement. The test suite exercises exactly the same protocol code that runs in deployment, only over a network that can be told to drop, delay and reorder messages at will.

Clients have their own communication channel, separate from the one used by Raft, via HTTP/JSON to the published

ports, over the endpoints described in Table I. The `Clerk` in the client library remembers the node it last got an answer from and, on `ErrWrongLeader`, advances to the next server and retries. Followers answer read and write requests with HTTP `421` and `ErrWrongLeader` rather than forwarding them, so leader discovery is entirely client-side. Every request carries a `(ClientId, Seq)` pair so that retried requests are answered from a cache instead of being applied twice.

The HTTP API also exposes a `/status` endpoint that reports the term, the observed leader, the commit, last-log and applied indexes, and the sizes of the Raft state and snapshot; the experiments in Section VII poll it to derive the recovery metrics.

Table I
HTTP ENDPOINTS EXPOSED BY EVERY NODE

Method	Path	Effect
GET	`/status`	node state: term, leader, indexes, state and snapshot sizes
GET	`/kv/{key}`	read the value of a key
PUT	`/kv/{key}`	set the key to the request body
POST	`/kv/{key}`	append the request body to the key

Finally, each node persists its durable state to a mounted volume. The raft side persists the current term, vote and log, and the kvserver side persists the key/value map and the response cache. For more details, see (Section III-D).

V. IMPLEMENTED ALGORITHMS

[TODO: Make explicit that these are the two complementary algorithms: leader election (theme 1) and consensus/replication (theme 2).**]**

A. Leader election and terms

[TODO: Randomized timeouts, voting (log up-to-date rule, one vote per term), conversion to follower on a higher term; role of the observed `leaderId`.**]**

B. Log replication and consensus

[TODO: `AppendEntries` with `PrevLogIndex/Term`, retry with `XTerm/XIndex/XLen`, `commitIndex` advanced only for entries from the current term (Figure 8 of the paper), snapshots and `InstallSnapshot`.**]**

C. Linearizable KV state machine

[TODO: Every operation (reads included) goes through the log; response cache keyed by `(ClientId, Seq)` for exactly-once application; replies matched by log index; snapshot of the map plus the cache.**]**

D. Reused code and modifications

[TODO: Mandatory citation: MIT 6.824 labs 2A–3B skeleton (`labrpc`, `labgob`, `porcupine`, tests) [2]; describe what was changed: the Raft/KV implementation, the `RPCClient` interface for real TCP, the on-disk `Persister`, the `/status` endpoint, the benchmark client and the experiment scripts.**]**

VI. CORRECTNESS PROPERTIES AND TESTS

The Raft paper [1] already proves that the protocol is safe, so we did not try to prove anything new. What we had to make sure of was that our code really follows the rules that proof depends on, and then test that it does. This section goes through the safety properties of Section III-E, shows where each one comes from in the implementation, and then describes the tests we used to check them.

A. Safety arguments

a) One leader per term: A node grants at most one vote per term, and it saves `votedFor` and `currentTerm` before replying, so not even a crash makes it forget a vote. A candidate needs votes from more than $N/2$ nodes, its own included. Two leaders in the same term would need two such majorities, and any two majorities of the same N nodes share at least one node, which would have had to vote twice.

b) Log Matching: A leader only appends to its own log, so it creates at most one entry per index in its term. Each `AppendEntries` RPC carries the index and term of the entry right before the new ones, and the follower refuses it unless it has that exact entry. Every accepted request therefore extends a prefix that the follower already shares with the leader, and by induction, if two logs have an entry with the same index and term, they agree on everything before it.

c) Committed entries are never lost: The main idea that supports this is the election restriction, which states that a node only votes for a candidate whose log is at least as up to date as its own, meaning its last entry has a higher term, or the same term and an index at least as high. A committed entry is stored on a majority and the new leader was voted in by a majority, so at least one of a leader's voters has the entry and would have refused a candidate without it. The paper turns this intuition into an induction over terms.

d) Same commands in the same order: `commitIndex` never decreases, and a single goroutine per node delivers entries to the service strictly in index order. Together with the two properties above, this means every replica applies the same command at each index, so all KV replicas go through the same sequence of states.

e) Linearizability: Every client operation, `Get` included, is turned into a log entry, and the server only replies after that entry is applied. The moment of application falls between the request and the reply, which is exactly the instant linearizability asks for. It also means that an old leader that was cut off cannot serve stale reads: it cannot commit anything, so its reads never return. The tricky part is retries. If a reply is lost, the `Clerk` sends the request again, possibly to another server, and the same command can end up in the log twice. Each server stores, for every client, the `Seq` and result of the last operation it applied, and a request whose `Seq` is not newer than that gets the stored result instead of being applied a second time. Since a `Clerk` has only one request in flight at a time, remembering the last one is enough. This table is saved in the snapshot, so it survives restarts and snapshot installs.

f) Liveness: Unlike safety, liveness depends on timing. Randomized election timeouts make repeated split votes unlikely, and once a leader can reach a majority, its heartbeats keep the followers from starting new elections while it retries `AppendEntries` until every entry is replicated.

B. Automated tests

The 6.824 labs' tests run every peer inside a single Go process, connected through `labrpc`, which gives the tester full control of the network. It can disconnect a node, split the cluster into two partitions, or crash a node and later restart it from a copy of what it had persisted. It can also make the network unreliable with artificial delays and packet loss.

What makes the Raft tests (2A–2C) strong is that safety is checked all the time, not only at the end. The tester records every entry that each peer applies, and fails immediately if two peers apply different commands at the same index, or if a peer applies an index before the previous one. Every time it looks for the leader, it also fails if it sees two leaders in the same term. So every Raft test, whatever it focuses on, is also checking that all replicas apply the same commands and that there is only one leader per term. The KV tests (3A–3B) look at the system from the outside instead, checking the values that clients get back. Every test also has a 120 s limit, so an implementation that stays safe but takes too long to recover also fails. Table II lists the tests by part and how long they took to execute; a description of each is given below.

Table II
AUTOMATED TESTS AND THEIR RUNNING TIME

Part	Focus	Tests	Time (s)
2A	leader election	2	10.1
2B	log replication	8	62.2
2C	persistence, Figure 8, churn	8	129.3
3A	KV service, linearizability	13	249.3
3B	snapshots	8	152.5
Ours	real TCP, HTTP and disk	1	0.7
Total		40	604.1

a) 2A – leader election: `InitialElection` checks that three nodes elect exactly one leader and agree on its term, and that the leader is still there two seconds later. `ReElection` disconnects the leader and expects a new one to be elected. The old leader coming back must not disturb the new one, with two of the three nodes disconnected no leader may be elected, and once a majority is back, one must be.

b) 2B – log replication: Here commands are submitted with `Start` and must be committed at the expected index by the expected number of nodes. `FailAgree` keeps committing with one follower disconnected and checks that it catches up when it comes back, while `FailNoAgree` checks that nothing commits when three of five followers are gone. In `Rejoin`, a disconnected leader accepts entries that can never commit, and when it reconnects it must drop them in favor of the new leader's log. `Backup` pushes this further: two minority partitions each pile up 50 uncommitted entries, and then nodes are reconnected in other combinations, so the leader has to overwrite long conflicting suffixes on its followers. `ConcurrentStarts` calls `Start` from several goroutines at once. Finally, `RPCBytes` and `Count` test for excessive communication, making sure that each command should be sent about once to each follower, an election may take at most 30 RPCs, and one idle second at most 60 RPCs.

c) 2C – persistence: The three `Persist` tests crash and restart nodes in different orders (all of them, only the leader, a follower while the others keep committing, a partitioned leader together with a follower), and a restarted node must come back with its term, vote and log. `Figure8` recreates the scenario behind the current-term commit rule: for 1000 iterations it gives a command to the leader and crashes it shortly after, usually before the entry commits, and restarts nodes whenever fewer than three are up. `Figure8Unreliable` does the same with disconnections instead of crashes, on an unreliable network, and turns on long reordering after 200 iterations. `UnreliableAgree` commits commands concurrently over the lossy network. The two `Churn` tests have three clients submitting commands while nodes are randomly disconnected, crashed and restarted every 700 ms; at the end, every value that a client saw committed must be in the final log of every node.

d) 3A – KV service: Most of these tests share the same routine, with five servers and three rounds of five seconds, in which each client appends to and reads its own key. Since each client is the only writer of its key, it knows what every `Get` must return, and any other value fails the test. After each round, every key is read back and each append must appear exactly once and in order, so both a lost write and a duplicate caused by a retry are caught. Options add more clients (five instead of one), an unreliable network, a partitioner that splits the servers into two random groups about once per second, and a restart of every server at the end of each round. `UnreliableOneKey` has five clients appending to the same key over the lossy network. `OnePartition` puts the leader on the minority side of a partition. The majority must keep serving requests, a `Put` and a `Get` sent to the minority must not complete, and the `Put` must go through once the partition heals. The `Get` on the minority side is the one that would catch an old leader serving stale reads.

e) 3B – snapshots: These tests set `maxraftstate` to 1000 bytes and, after each round, require the persisted Raft state to be at most eight times that, which means the log is really being trimmed. In `SnapshotRPC`, one server falls behind in a minority while the other two commit 50 writes and compact their logs. The partition then changes so that the lagging server is needed for a majority, and the only way it can catch up is through `InstallSnapshot`, since the leader has already thrown away the entries it is missing. `SnapshotSize` requires snapshots smaller than 500 bytes after 800 operations on a single key. The other six tests repeat the 3A combinations of faults, with up to 20 clients, with snapshots turned on.

f) 3A + 3B – linearizability: There are two *Linearizable* tests, one in 3A and one in 3B. Fifteen clients send random

`Put`, `Append` and `Get` requests on shared keys to seven servers, with lossy network, partitions and restarts all active, and the start and end time of every operation is recorded. Porcupine [3] then searches for a single order of all the operations that respects real time (if one operation returned before another started, it must come first) and that explains every value returned by a `Get`. If no such order exists, the history is not linearizable and the test fails.

g) Our own test.: All inherited tests use `labrpc` and the in-memory persister, so none of them runs the code that is used in the containers. To cover it, we wrote `TestHTTPAndTCPCluster`, which starts three nodes wired the same way as in the deployment: Raft over `net/rpc` on real TCP sockets, state saved to files on disk, and clients talking HTTP. It runs a `Put`, an `Append` and a `Get` through the `Clerk` and then checks three things: followers answer a write with HTTP 421 instead of applying it, `/status` reports the same leader on every node, and every node has *lastApplied* $\leq$ *commitIndex* $\leq$ *lastLogIndex*. This is only a smoke test without faults; the experiments of Section VII are what exercise the deployment under faults.

Results: All 40 tests pass.

VII. Experimental Methodology

Table III
Execution environment

Item	Value
Hardware	**[TODO:** CPU, cores, RAM]
Operating system	**[TODO:** distribution, kernel]
Docker / Compose	**[TODO:** versions]
Go	**[TODO:** version]
Nodes	3, 5, 7 containers (same host)
Load	8 clients, 50 ops/client
Seeds	5–10 per configuration
Warm-up	1 discarded run (sweep)

[TODO: Describe the workload (50/50 read/write mix, 5 keys, `kvbench`), the per-run key namespace, and the fact that throughput is measured before the linearizability check.] **[TODO:** Metrics: throughput (ops/s), latency p50/p95/p99, time-to-elect, service recovery time (TTR-service), replica recovery time (TTR-replica), attempts per operation and `ErrWrongLeader` rate. Justify each one.] **[TODO:** Parameters varied at ≥ 3 levels: $N \in \{3, 5, 7\}$; delay/loss via `tc netem` (0, 50 ms, 100 ms; 0%, 2%, 5%); number of clients.] **[TODO:** Fault scenarios: leader crash, leader partition, full cluster restart.] **[TODO:** Reproducibility: `scripts/experiment.py` with recorded seeds, raw CSVs in `scripts/experiments/`, analysis in `scripts/analysis.ipynb`. Exact commands.]

VIII. Results

[TODO: Replace the values below with the final version and generate the figures with `scripts/analysis.ipynb`.]

Table IV
Throughput and latency (median over seeds)

Scenario	ops/s	p50	p95	p99
Normal	1633	4.5	7.4	8.8
`netem` 50 ms/2%	41	155	307	418
Leader crash (2 nodes)	2038	2.7	8.0	21.9
Partition (after heal)	1227	2.6	6.4	184

Table V
Recovery times (mean $\pm$ std)

Scenario	election	TTR-service	TTR-replica
Leader crash	507 ± 114 ms	1868 ± 32 ms	468 ± 124 ms
Partition	349 ± 185 ms	1632 ± 404 ms	not applicable

A. Normal operation

[TODO: Describe throughput and percentiles; explain that the 2% retry in the baseline is one extra attempt per client to discover the leader (8/400), not steady-state retrying.]

B. Delay and loss

[TODO: Discussion: throughput drop and p99 increase; why the ceiling is dominated by the election timeout and retransmissions; linearizability is preserved.]

C. Leader crash

[TODO: Time-to-elect $\approx$ election timeout; TTR-service dominated by the client HTTP timeout ($\sim$2 s); TTR-replica measures recovery via log/snapshot.]

D. Partition and the disruptive-server problem

[TODO: Inflated terms, churn, attempts per operation; explain that without PreVote the isolated node exiles itself; treat it as a limitation and future work.]

E. Scalability

[TODO: Results with $N = 3, 5, 7$: throughput and latency; discuss why the leader serializes and where replication cost would show up (message counts).]

IX. Limitations and Threats to Validity

[TODO: Containers on one host are not independent failure domains; variability from short runs; no batching; no PreVote; no message/byte counters; Docker Desktop environment.]

X. Conclusion and Future Work

[TODO: Recap: two algorithms, correctness under faults, performance cost. Future work: PreVote/check-quorum, batching, lease-based reads, incremental snapshots, message accounting, execution across multiple machines.]

REFERENCES

[1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*. Philadelphia, PA: USENIX Association, 2014, pp. 305–319. [Online]. Available: https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro

[2] R. Morris, F. Kaashoek *et al.*, "6.824 distributed systems: Labs 2 and 3," MIT Computer Science and Artificial Intelligence Laboratory, 2020, skeleton for `labrpc`, `labgob`, `porcupine` and the test suite reused in this work. [Online]. Available: https://pdos.csail.mit.edu/6.824/2020

[3] A. Athalye, "Porcupine: A fast linearizability checker for go," GitHub repository, 2017. [Online]. Available: https://github.com/anishathalye/porcupine

[4] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. distributed-systems.net, 2023. [Online]. Available: https://www.distributed-systems.net/index.php/books/ds4/